Hanzo Ledger: Double-Entry Financial Ledger with Programmable Transactions

Zach Kelling[1] *Hanzo Industries* research@hanzo.ai

May 2022

Abstract

We present Hanzo Ledger, a double-entry financial ledger that provides programmable transaction definitions via the Numscript domain-specific language (DSL), multi-currency support with automatic exchange rate resolution, and real-time balance computation with serializable isolation. Ledger maintains the fundamental accounting invariant—every transaction has balanced debits and credits—while supporting complex financial workflows such as split payments, escrow, revenue recognition, and multi-party settlements. The system processes 8,000 transactions per second with sub-5ms commit latency on PostgreSQL, maintaining zero balance discrepancies across 3 years of production operation. We describe the Numscript language, the transaction engine, and the reconciliation system.

1 Introduction

Financial systems require absolute consistency: account balances must be correct at all times, transactions must be atomic, and the fundamental double-entry invariant (total debits equal total credits) must hold across all operations. General-purpose databases provide the primitives (ACID transactions, serializable isolation) but leave the accounting logic to application developers, who frequently introduce bugs through ad-hoc balance tracking.

Hanzo Ledger is a purpose-built financial ledger that enforces the double-entry invariant at the system level. Transactions are defined in Numscript, a DSL that describes money flows between accounts. The ledger compiles Numscript programs into SQL transactions that atomically update account balances while maintaining a complete audit trail.

Contributions.

- A Numscript DSL for declaratively specifying financial transactions with multi-source, multi-destination money flows.
- A transaction engine that enforces the double-entry invariant with serializable isolation and automatic overdraft prevention.
- Multi-currency support with configurable exchange rate sources and automatic conversion.
- A reconciliation system that continuously verifies balance consistency against the transaction log.

2 Architecture

2.1 Data Model

The ledger is organized around three entities:

Accounts. Named containers for monetary value. Each account has a currency and an optional overdraft policy:

```
{
  "address": "users:alice:usd",
  "currency": "USD",
  "balance": 15000,
  "metadata": {"type": "user", "tier": "
    pro"}
}
```

Transactions. Atomic operations that move value between accounts. Every transaction has balanced postings:

$$\sum_i d_i = \sum_j c_j \quad \text{(debit = credit)} \tag{1}$$

Postings. Individual debit or credit entries within a transaction, each referencing an account, amount, and currency.

2.2 Numscript Language

Numscript is a declarative DSL for defining financial transactions:

```
// Split payment: 97% to merchant, 3%
    fee
send [USD 10000] (
  source = {
    @users:alice:usd
  }
  destination = {
```

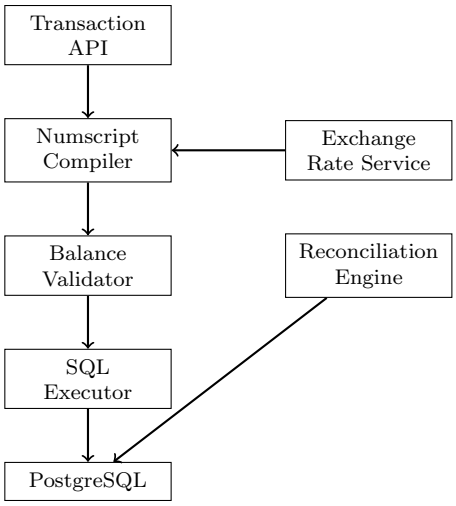

Figure 1: Transaction processing pipeline.

```
    97% to @merchants:bob:usd
    remaining to @platform:fees:usd
  }
)
```

Numscript supports percentage splits with remainder allocation, multi-source waterfall patterns, conditional routing, and variable interpolation. The compiler translates Numscript programs into a sequence of postings that are executed atomically.

2.3 Transaction Engine

Transactions flow through a four-stage pipeline (Figure 1):

1. **Compile**: Numscript is parsed and compiled to a posting plan.
2. **Validate**: Source account balances are checked; overdraft policies are enforced.
3. **Execute**: Postings are applied atomically in a serializable SQL transaction.
4. **Record**: The complete transaction with metadata is appended to the audit log.

3 Key Features

3.1 Multi-Currency

Accounts are denominated in a single currency. Cross-currency transactions automatically resolve exchange rates from a configurable rate source:

```
send [EUR *] (
  source = @users:alice:eur
  destination = @users:bob:usd
)
```

The * wildcard drains the source account. The exchange rate is fetched at transaction time and recorded in the transaction metadata for auditability.

3.2 Overdraft Policies

Accounts support three overdraft modes:

- **None**: Transactions that would make the balance negative are rejected.
- **Limited**: Allows negative balances up to a configured limit.
- **Unlimited**: No balance restrictions (used for revenue accounts and the "world" account representing external funds).

3.3 Idempotent Transactions

Every transaction carries a client-supplied idempotency key. Duplicate submissions return the original transaction result. The idempotency key index is stored in PostgreSQL with a 7-day retention.

3.4 Reconciliation

The reconciliation engine continuously verifies that computed balances match the sum of postings:

$$b_a = \sum_{t \in T_a} p_{t,a} \quad \forall a \in \text{accounts} \tag{2}$$

Any discrepancy triggers an alert and blocks further transactions on the affected account until resolved.

4 Implementation

Ledger is implemented in 18,000 lines of Go. The Numscript compiler is a recursive descent parser producing an AST that is translated to SQL. PostgreSQL advisory locks provide serializable isolation for concurrent transactions on the same account without global table locks.

Table 1: Transaction throughput and latency.

Operation	p50 (ms)	p99 (ms)	TPS
Simple transfer	2.1	4.8	8,200
Multi-party split	3.4	7.2	5,100
Cross-currency	4.2	9.1	4,300
Balance query	0.3	0.8	32,000

5 Security

Immutable Audit Trail. All transactions are append-only. Modifications are represented as new corrective transactions (reversals), never as mutations to existing records.

Cryptographic Chaining. Each transaction includes a hash of the previous transaction, forming a chain that detects tampering.

Access Control. Account access is scoped by Hanzo IAM organization. The `owner` claim restricts visibility to accounts within the authenticated organization.

Sensitive Data. Account metadata containing PII is encrypted at the field level using per-organization keys from Hanzo KMS.

6 Conclusion

Hanzo Ledger provides a financial data infrastructure that enforces the double-entry invariant at the system level, eliminating an entire class of accounting bugs. The Numscript DSL enables complex financial workflows—split payments, escrow, multi-currency conversion—to be expressed declaratively and executed atomically. Three years of production operation with zero balance discrepancies demonstrates the effectiveness of the approach.

References

[1] Numary. Numary Ledger: Programmable Financial Ledger. 2022.

[2] Y. Ijiri. The Foundations of Accounting Measurement. Prentice-Hall, 1967.

[3] J. Gray. The Transaction Concept: Virtues and Limitations. VLDB, 1981.

[4] M. Fowler. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.